

OCEANEMBED

Satellite Embedding-Based Deep Learning for Reconstruction of Subsurface Ocean Temperature

SIH26066 · Ministry of Earth Sciences (INCOIS) · Smart India Hackathon 2026

Deliverable	Round 1: idea PPT + team-narrated demo video (no prototype required)
Portal deadline	20 September 2026 · internal cut-off 15 September 2026
Build window	5 days
Team	3 people, 3 Claude accounts
Prototype due	Grand Finale, December 2026 (36-hour software sprint)
Target of these 5 days	A real end-to-end slice, so the Round-1 video shows genuine numbers

1. The mission, in one paragraph

Satellites see only the ocean’s skin. What actually drives cyclones, fisheries and marine heatwaves is the temperature hundreds of metres down — measured today by a few thousand drifting ARGO floats, which are far too sparse for continuous basin-scale coverage. OceanEmbed compresses surface satellite variables into a learned latent representation and decodes it into the full vertical temperature profile, then proves the result against ARGO measurements the model has never seen.

THE FRAME THAT MATTERS. Round 1 is judged on an idea PPT and a demo video — not on a finished product. These 5 days exist to make that video show real numbers instead of a mockup. That is both the winning move for September and the head start for December.

Four corrections to the earlier ChatGPT plan

These were verified against the official problem statement. Each one changes what you build.

What the earlier plan said	What the official PS actually requires
ARGO is the training ground truth	GLORYS12V1 reanalysis is the TRAINING TARGET. Independent ARGO is the VALIDATION set. Field-to-field, not point regression against floats.
Start with 5 depths, extend to 2000 m	15 named depths, 0–1000 m: 0, 5, 10, 20, 30, 50, 75, 100, 125, 150, 200, 300, 500, 700, 1000. Nothing below 1000 m.
Start with an MLP and call it an encoder-decoder	The satellite embedding engine is a NAMED deliverable — CNN / ViT / autoencoder / GNN / attention. An MLP is a valid baseline, not a valid answer.
Pick any region	Fixed domain: 5°N–30°N, 45°E–105°E, 0.25°, daily. PoC over Bay of Bengal / Arabian Sea is explicitly sanctioned.

The scope we are actually building

Region	Bay of Bengal box: 5–22°N, 80–95°E (0.25° → roughly 68 × 60 grid cells)
Time	2021–2022 daily for training · 2023 held out entirely (temporal holdout, never a random split)
Depths	All 15 PS levels. Output dimension is free — do not reduce it, it is a direct spec match.
Inputs	SST, SLA, SSS, surface currents (u,v), surface winds (u,v) + lat/lon/day-of-year encodings
Target	GLORYS12V1 thetaso, regridded to 0.25° and interpolated onto the 15 levels
Validation	Raw ARGO profiles inside the box during 2023, pulled from Ifremer ERDDAP

The closing claim

“OceanEmbed learns a compact latent representation of the sea surface and decodes it into a 15-level temperature profile. On a fully held-out year it achieves RMSE X°C, and against N independent ARGO profiles it has never seen, Y°C. The architecture is domain-agnostic and scales to the full 5–30°N / 45–105°E basin.”

2. Hour 0 — before Day 1 begins

Two of these are outside your control and sit on the critical path. Do them first, today.

BLOCKER FOUND ON THIS MACHINE. You are running Python 3.14.3, and torch, xarray, netCDF4 and dask are not installed. The scientific stack is unreliable on 3.14 this early — PyTorch and netCDF wheels lag new releases. Build inside a Python 3.11 or 3.12 virtual environment. Forcing 3.14 can burn a full day of the five.

Confirmed hardware

GPU	NVIDIA RTX 4050 Laptop, 4 GB VRAM — sufficient for the patch-CNN with small batches
Disk	D: 69.5 GB free · C: 331.8 GB free. Enough for the BoB subset; keep downloads tight.
Verdict	GPU branch of the plan applies. 128-d embedding, 32×32 patches, batch size 64–128.

Checklist

1. Register for Copernicus Marine at marine.copernicus.eu (free, usually instant). This gates the training target — if it is not working within an hour, raise it immediately.
2. Create the environment and install the stack:

```
py -3.12 -m venv .venv && .venv\Scripts\activate
pip install torch --index-url https://download.pytorch.org/whl/cu121
pip install xarray netcdf4 zarr dask scikit-learn fastapi uvicorn
pip install matplotlib copernicusmarine pyyaml requests
```
3. Verify the GPU is visible:

```
python -c "import torch; print(torch.cuda.is_available())" # must print True
```
4. Start the GLORYS download running in the background immediately. It is the largest file and everything else waits on it.
5. Create the GitHub repo and push the skeleton + CLAUDE.md, so the other two accounts can clone and start. See Section 5.

Data sources, with auth risk flagged

Variable	Product	Auth	Risk
TARGET: subsurface T	CMEMS GLORYS12V1 — cmems_mod_glo_phy_my_0.083deg_P1D-m, var thetao	CMEMS	Large download — start first
SST	NOAA OISST v2.1, 0.25° daily (NCEI)	none	Low
SLA + geostrophic currents	CMEMS SEALEVEL_GLO_PHY_L4_MY_008_047	CMEMS	Low
SSS	CMEMS MULTIOBS_GLO_PHY_S_SURFACE_*	CMEMS	Low
Surface currents (total)	CMEMS MULTIOBS_GLO_PHY_MYNRT_015_003	CMEMS	Low
Winds (u10, v10)	CMEMS ASCAT L4 WIND_GLO_PHY_L4_*	CMEMS	Do NOT use ERA5/CDS — queue can take days
INDEPENDENT ARGO	Ifremer ERDDAP ArgoFloats tabledap, CSV over plain HTTPS	none	Low — needs no account

Confirm every CMEMS dataset ID with `copernicusmarine describe --contains <keyword>` before writing a fetcher. IDs get versioned, and the ones above are from memory, not a live catalogue read.

CUT RULE. If winds are the only thing still blocking you at the end of Day 1, ship with 5 input channels instead of 7 and say so on the slide. Nobody will fail you for a missing variable. A missing Day 2 will.

3. The five days

DAY 1 Data acquisition and harmonisation

Owner: P3 leads, P1 assists — this is the only day that can kill the project

- Write configs/default.yaml — region, date range, the 15 depths, variable list. Every other module reads this and nothing hardcodes it.
- One fetcher per source in oceanembed/acquire/. Each independently runnable and idempotent (skip if the file already exists), so a dropped connection costs minutes, not a day.
- prep/regrid.py — put everything on the same 0.25° grid and the same daily time axis. GLORYS goes $1/12^\circ \rightarrow 0.25^\circ$, then `.interp(depth=DEPTHS)` onto the 15 levels.
- prep/harmonize.py — stack into one analysis-ready xarray Dataset at `data/processed/bob_2021_2023.zarr`, dims (time, depth, lat, lon).
- In parallel and non-blocking: P2 freezes the API contract and builds the entire dashboard against a hand-written mock JSON. P2 must never be blocked waiting on P1.

DAY 1 GATE. Do not start Day 2 until all four are true — (1) the zarr loads and `isel(time=0)` plots a sane Bay of Bengal SST map; (2) no all-NaN days, land mask consistent across variables; (3) a GLORYS profile at a random ocean point is physically plausible, roughly 28°C at the surface and $8\text{--}12^\circ\text{C}$ at 1000 m; (4) 2023 is present and completely untouched.

DAY 2 Baselines — the honest number to beat

Owner: P1

- `dataset.py` — sliding 32×32 patches. One sample = a patch of surface channels $\rightarrow$ the 15-level profile at the patch centre. Normalise per channel using training-split statistics only.
- Build three baselines, in this order:
 - Depth-wise climatology (mean profile per month) — the number that proves the model learned something real rather than memorising the seasonal cycle.
 - Per-pixel linear regression, 7 surface variables $\rightarrow$ 15 depths.
 - Per-pixel MLP — no spatial context, which is exactly what the CNN must beat to justify itself.
- `evaluate.py` writes `results/metrics.json` — RMSE, bias and correlation, per depth level plus overall. Establish this file format now; every later stage only appends to it.

DAY 2 GATE. A `results/metrics.json` containing all three baselines, and a per-depth RMSE plot. If the linear baseline already beats climatology, your data pipeline is sound.

DAY 3 The OceanEmbed model — the actual deliverable

Owner: P1

- Architecture: surface patch ($7 \times 32 \times 32$) $\rightarrow$ `encoder.py`, 3–4 conv blocks with GroupNorm and global pooling $\rightarrow$ the 128-d ocean embedding $\rightarrow$ `decoder.py`, a depth-conditioned FiLM-style MLP $\rightarrow$ temperature at 15 depths.
- Train with masked MSE, skipping land and NaN cells. Weight the loss by depth, or normalise per level, so the thermocline is not drowned out by the trivially well-fit surface.
- Checkpoint to `oceanembed-v1.pt` and record the config hash inside the checkpoint, so no result is ever orphaned from the settings that produced it.
- Run the model convolutionally over the whole box to produce the full (15, 68, 60) 3D field for a held-out date. This is the visual money shot for the video.

- Stretch, only if training finished early: t-SNE or UMAP of the embedding coloured by month or mixed-layer depth. If the latent space organises itself by ocean regime, that is direct evidence the embedding is doing real work — which is precisely what the PS is asking for.

DAY 3 GATE. OceanEmbed beats all three baselines on held-out 2023, per depth. If it does not, ship the best baseline and say so honestly. An honest MLP that beats climatology is a far better Round-1 story than a CNN with invented numbers.

DAY 4 Independent ARGO validation, API, dashboard

Owner: P1 → ARGO · P2 → API + dashboard · P3 → ops slice

- ARGO validation (P1 — the single highest-value task of the whole week). Pull 2023 ARGO profiles inside the box from Ifremer ERDDAP. Quality-control on TEMP_QC in (1,2), interpolate each profile onto the 15 levels, match to the nearest grid cell and day, run the model, compare. Append to metrics.json: n_profiles, overall RMSE/bias/correlation, per-depth RMSE, plus 3–4 individual profile overlay plots.
- That overlay figure — your prediction drawn on top of a real float measurement it never saw — is the most persuasive artefact in the entire submission. It answers the PS's own stated validation requirement literally.
- API (P2): FastAPI serving /predict, /profile, /validate, /health, /metrics. /validate returns real numbers read straight out of metrics.json.
- Dashboard (P2): swap the mock for the live API. Map of the box coloured by temperature at a selectable depth, a vertical profile panel, an ARGO overlay toggle, and the metrics strip.
- Ops slice (P3, roughly 2 hours, and the first thing to cut): prometheus_client on the FastAPI app exposing oceanembedded_predictions_total, oceanembedded_inference_latency_seconds and oceanembedded_model_rmse; docker-compose with Prometheus and Grafana; one board; one alert rule for API-down.

DAY 4 GATE. n_profiles > 0, per-depth ARGO RMSE written to metrics.json, and the profile overlay plots look physically sensible to the eye.

DAY 5 Evidence, PPT, video — the actual submission

Owner: All three

- Figures (P1): per-depth RMSE bar chart; predicted-vs-ARGO profile overlays; a depth-slice temperature field; scatter of predicted vs true at three representative depths.
- PPT (P3), in the official SIH 2026 template: problem → why surface signals carry subsurface information (thermocline displacement, mesoscale eddies, vertical mixing) → the embedding architecture → real results → validation against independent ARGO → the scale-up path to the full basin → the December roadmap.
- Video (all three, roughly 2 hours): run inference → show the 3D field → click a point → the profile appears → overlay the real ARGO profile → show the metrics → if the ops slice survived, kill the API container, show the alert fire, restart, show recovery.
- Keep the rest of the day as buffer. Something will break on Day 5.

SIH RULE — DISQUALIFYING IF BROKEN. The demo video narration must be delivered by team members. Not AI-generated audio. Not a synthesised voice. Record it yourselves.

4. Ownership, cut order, non-negotiables

Who owns what

Only P1 sits on the critical path. P2 and P3 are structured so they can never be blocked waiting for the model.

	Days 1–2	Days 3–4	Day 5
P1 — ML	Regrid, harmonise, baselines	OceanEmbed model, ARGO validation	Figures, technical slides
P2 — backend / UI	Freeze API contract, build dashboard on mock data	Wire the real API, polish dashboard	Video screen-recording
P3 — data / ops / pitch	All acquisition, CMEMS auth, ARGO pull	Docker, ops slice	PPT, narration, submission

Cut order — for when, not if, a day slips

Cut strictly from the bottom up. Never cut upward past the line.

7	Grafana / Prometheus / any alerting — cut first, without hesitation
6	Docker
5	Embedding t-SNE / UMAP visualisation
4	Map view (keep the single-point profile view)
3	FastAPI (fall back to a notebook-driven demo)
—	NEVER CUT BELOW THIS LINE
2	Independent ARGO validation
1	Real data and a real model beating a real climatology baseline

Non-negotiables

1. No fabricated numbers. Every metric traces to results/metrics.json, written only by evaluate.py. Claude writes the code; Claude does not invent the results. This is the single biggest credibility risk in an AI-assisted build.
2. 2023 stays untouched until Day 4 evaluation. One accidental leak invalidates every number you report.
3. All 15 PS depth levels, and the exact PS region box in the config — even though the PoC trains on the Bay of Bengal subset.
4. Video narration by team members. SIH rule, disqualifying.
5. Config-driven, never hardcoded. One default.yaml; region, depths and dates change in exactly one place. This is what makes “scales to the full basin” a true statement rather than a claim.
6. Give Claude bounded tasks with the architecture pre-specified — one module at a time, each with a test. Never “build OceanEmbed”.

5. Working with three Claude accounts

Three accounts give you three parallel workers, but they share no memory. Nothing one Claude learns is visible to the others. Everything below exists to solve exactly that problem: the git repository, not any conversation, is the shared brain.

5.1 Account map

Account	Person	Owns these directories	Never touches
A (this one)	P1 — ML lead	oceanembed/prep/, oceanembed/models/, train.py, evaluate.py, configs/	api/, dashboard/, docker
B (Pro)	P2 — backend/UI	oceanembed/api/, dashboard/	models/, prep/, train.py
C (Pro)	P3 — data/ops/pitch	oceanembed/acquire/, docker- compose.yml, prometheus/, docs/	models/, api/

THE ONE RULE THAT MAKES THIS WORK. No two Claude accounts ever edit the same file. Every merge conflict in a 5-day sprint costs an hour you do not have, and conflicts in AI-generated code are unusually nasty because both sides look plausible.

5.2 Set-up, in order

- Account A creates the GitHub repo and pushes the skeleton on Day 0 — folder structure, CLAUDE.md, configs/default.yaml, the frozen API contract, and the metrics.json schema. Nobody starts before this exists.
- B and C clone it. Because CLAUDE.md is committed in the repo root, Claude Code auto-loads it in all three sessions — that is what keeps three separate models building one coherent system.
- Each person works on their own branch: feat/model, feat/api, feat/data.
- One integration checkpoint per day at a fixed hour. Everyone merges to main, then Account A runs the smoke test. Never merge silently mid-day.

5.3 Freeze the contracts on Day 0

These three artefacts are what let three people build in parallel without waiting for each other. Write them before any real code exists, and treat changes to them as a team decision, never a solo one.

Contract	What it fixes
configs/default.yaml	Region, dates, the 15 depths, variable names, normalisation stats path. Everyone reads it; nobody hardcodes around it.
The API contract	POST /predict → { profile: [{depth, temperature}], model_version, rmse }. P2 builds the entire dashboard against this shape with mock data, long before a model exists.
results/metrics.json schema	Field names for RMSE, bias, correlation, per-depth arrays and n_profiles. The PPT, dashboard and video all read from this one file.

5.4 The handoff file — the fix for zero shared memory

A Claude session that ends takes its context with it. Tomorrow’s session, on the same account, starts blind. So end every working day by having your Claude append to docs/HANDOFF.md:

- What was finished, and where the code for it lives
- What is half-done, and precisely where you stopped
- What broke, what you tried, and what you ruled out — this saves the most time of anything on the list

- What the next session should pick up first

Commit it. Next morning, open the session with “read docs/HANDOFF.md and continue”. This one habit is worth more than any other coordination trick here.

5.5 Pro plan practicalities

- Pro accounts have usage limits. Use Sonnet for bulk code generation and boilerplate; save the stronger model for architecture decisions and hard debugging.
- If an account hits its limit mid-task, the work is already in git — another account can pull the branch and continue. Commit early and often, precisely so this is survivable.
- Stagger the heavy sessions across the day rather than all three running flat out in the same hour.

5.6 How to prompt, and how not to

Do this	Not this
“Implement oceanembed/models/encoder.py exactly as specified in CLAUDE.md §3. Add a test that asserts the output shape is (B, 128).”	“Build the OceanEmbed model.”
One module per request, each with a test, each committed before the next.	A whole subsystem in one prompt, reviewed at the end.
Point every account at CLAUDE.md as the authority.	Pasting Account A’s answer into Account B as if it were a spec. The repo is the source of truth, not a transcript.
“Run evaluate.py and report what it printed.”	“What RMSE do you think we would get?”

THE FAILURE MODE TO WATCH FOR. Three Claudes, each independently redesigning the same module because none of them can see the others’ work. The countermeasures are: file-level ownership, contracts frozen on Day 0, one daily merge, and CLAUDE.md committed as the single shared authority. Skip these and three accounts will be slower than one.

5.7 Daily rhythm

Morning	Each person opens their session with “read CLAUDE.md and docs/HANDOFF.md, then continue”. Pull main first.
During	Work only inside your own directories. Commit every working module. Push to your own branch.
Evening	All merge to main. Account A runs the smoke test end-to-end. Everyone appends to HANDOFF.md.
If blocked	Say so in the group within 30 minutes. In a 5-day build, a silent blocker is the most expensive thing that can happen.

6. After Round 1 (October – December, if selected)

Full 5–30°N / 45–105°E domain · ViT and GNN embedding variants benchmarked against the CNN · salinity as a second decoder head · uncertainty estimates · the full Sentinel observability layer on a real cloud VM, where the “continuous basin-scale service needs monitoring” argument finally earns its place · deployment hardening for the 36-hour Grand Finale sprint.